11/06 CSV에서 PCAP으로

CSV 입력의 한계를 확인하고, 패킷 원본에서 feature를 직접 만드는 실험으로 전환

이전 실험	확인한 문제	이번 보고
CICFlowMeter CSV 벡터 입력 XGB · TabPFN SOTA · 제안 모델 평가 시나리오 표현 시도	분류 성능 부진 feature가 같은데 라벨이 다른 행 few-shot에서도 일부 공격의 성능이 높아 개선 여지 부족	CSV 대신 PCAP 사용 netFound로 feature 네트워크 데이터용 transformer 인코더

검증 질문 패킷 기반 feature로 앞의 문제가 개선되는가

문제1 CIC-IDS 데이터의 동일 feature·상이 라벨

실험망에서 PCAP을 수집하고, CICFlowMeter가 패킷을 flow로 묶어 미리 정한 통계를 계산

실험망	→	PCAP	→	CICFlowMeter	→	CSV
정상·공격 통신		시간순 원시 패킷		flow 묶기 · 고정 통계 계산		통계 벡터 + 라벨

고정 통계 flow마다 같은 계산식 78개 · 지속 시간 · 보낸·받은 패킷 수 · 패킷 길이 평균 · 패킷 간격 평균 · FIN 수

같은 feature에 다른 라벨이 생기는 경로

① flow 경계 결함 (주원인)	→	② 패킷이 매우 적은 flow	→	③ 라벨은 원래 flow를 따름
첫 FIN에서 끊고 RST 무시 남은 ACK·FIN이 별도 flow 이런 flow가 CIC2017의 25.9%		남은 패킷 1~2개로 계산한 통계 값이 쉽게 일치		같은 모양의 꼬리 flow에 정상과 공격 라벨이 혼재

근거 Engelen 2021 · Liu 2022 · PCAP 재구성에서 꼬리 flow를 빼면 통계 충돌 6,367 → 69

데이터	통계가 모두 같은 flow의 예	붙어 있는 라벨
CIC2017	목적 포트 80 · 1 µs · 보낸 2 · 받은 0	BENIGN 2행 · DoS Hulk 3,360행
CIC2018	목적 포트 21 · 2 µs · 보낸 1 · 받은 1	SlowHTTPTest 62,024행 · FTP 87,297행 · SSH 10행

문제1 CIC2017·2018 CSV의 class별 충돌

원본 CSV 전체 행 기준 · 통계가 완전히 같은데 class가 다른 묶음에 속한 행 · 행 순서는 CIC2017 행 수 내림차순

class	CIC2017 행	충돌 행	비율 %	CIC2018 행	충돌 행	비율 %
benign	2,273,097	736	0.03	13,484,708	125,771	0.93
dos	252,661	5,225	2.07	654,300	77,277	11.81
portscan	158,930	1,053	0.66	–	–	–
ddos	128,027	6	0.00	1,263,933	0	0.00
bf	13,835	0	0.00	380,949	162,479	42.65
web	2,180	0	0.00	928	5	0.54
bot	1,966	0	0.00	286,191	0	0.00
inf	36	0	0.00	161,934	41,279	25.49
heartbleed	11	0	0.00	–	–	–
합계	**2,830,743**	**7,020**	**0.25**	**16,232,943**	**406,811**	**2.51**

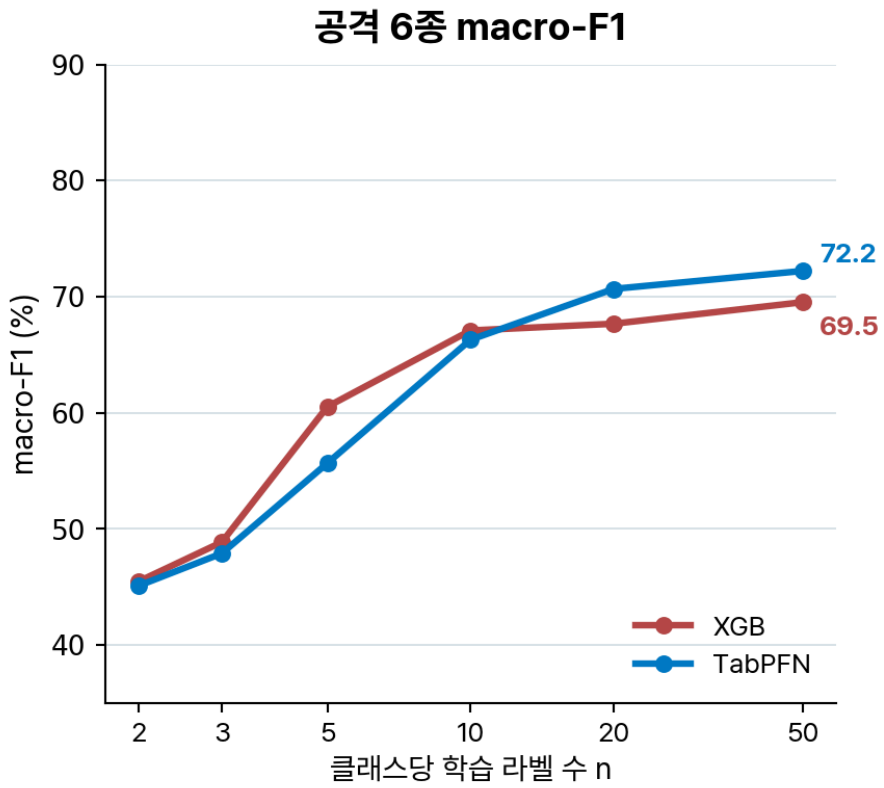
CIC2018은 bf 42.65% · inf 25.49% · dos 11.81%로 공격 class 충돌이 큼 · CIC2017은 dos 2.07%가 가장 큼

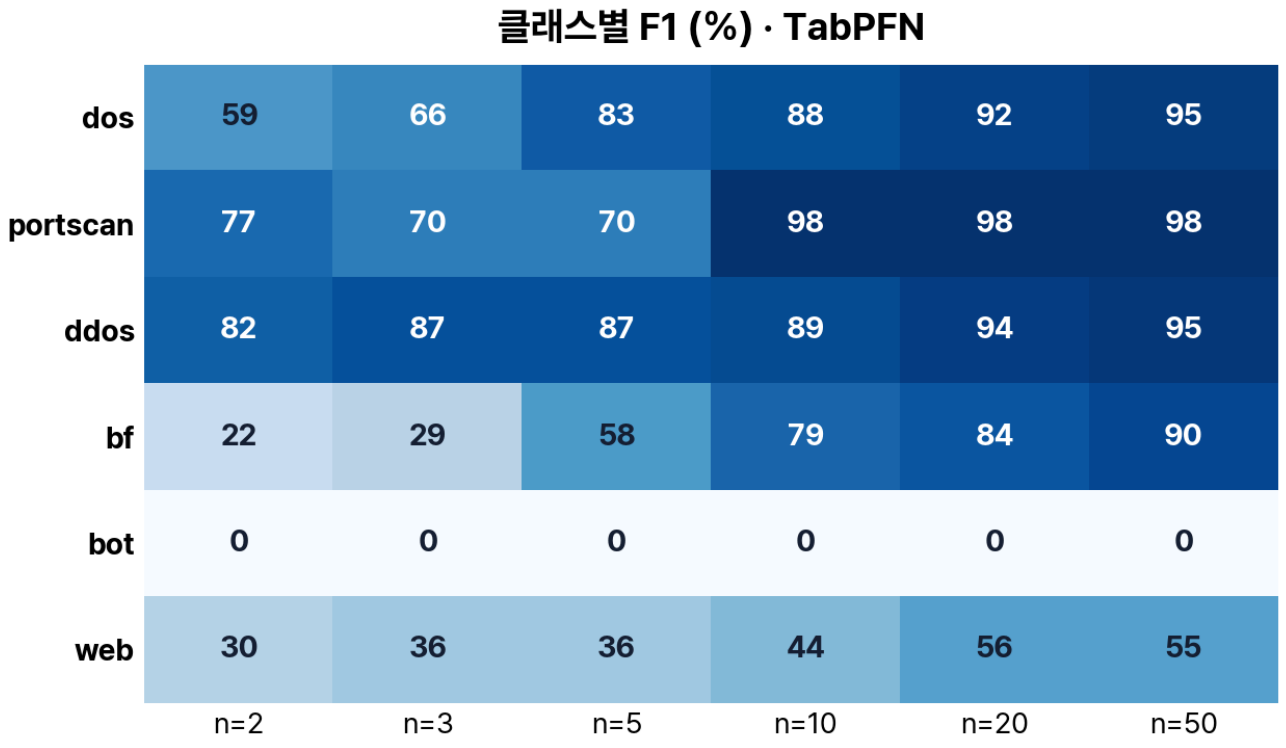
PCAP 크기 때문에 실험은 CIC2017로 진행 (CIC2018 450 GB, CIC2017 51 GB)

충돌 행은 CIC2018 2.51%, CIC2017 0.25%이고, CIC2018은 공격 class에 크게 집중

문제2 few-shot에서도 일부 공격은 이미 높은 성능

CIC2017 CSV · 공격만 학습·평가 · 클래스당 라벨 n개 · seed 5개 평균





n=20에서 portscan 98% · ddos 94% · dos 92% · n=50에서 98% · 95% · 95%

반면 web은 n=50에서도 55%, bot 0%는 CSV 라벨 결함 · 평균 성능만으로는 난이도 판단 불가

PCAP에서 feature까지

라벨은 feature 계산에 쓰지 않음 · PCA를 학습 데이터로만 맞추기 위해 시간순 분할이 PCA보다 먼저

① PCAP

CIC2017 월~금
하루 1개, 파일 5개
51 GB

② flow로 묶기

프로토콜·IP·포트 쌍이 같은 패킷
RST · 양쪽 FIN 뒤 ACK
새 SYN · 120초에서 종료

③ 라벨 붙이기

공격자·피해자 IP인가
공격 시간창 안인가
공격 행위 증거가 있는가

④ 시간순 분할

날짜·공격별로
앞 70% 학습
다음 15% · 뒤 15% 평가

⑤ 토큰화

IP 헤더 길이·TOS·전체 길이·flag·TTL
TCP flag·window·seq·ack·urgent
IP·포트·시각·payload 제외

⑥ netFound

토큰열 → 512차원
가중치 고정

⑦ PCA

512차원 → 64차원
학습 데이터로 맞춤

⑧ 평가

패킷을 flow로 묶고 라벨을 붙인 뒤, 헤더 토큰을 인코더에 넣어 flow당 64차원 feature를 생성

패킷의 생김새

패킷은 네트워크로 오가는 데이터 조각 하나 · 아래는 월요일 사내 PC가 웹서버로 보낸 실제 패킷 151바이트



이더넷 14 B

바로 옆 장비까지
전달하기 위한 표

보낸 장비 번호
받는 장비 번호

IP 헤더 20 B

어느 컴퓨터로 갈지 적는 칸

보낸 주소 192.168.10.9
받는 주소 69.31.33.224
전체 길이 137 B
TTL 128
프로토콜 6번 (TCP)

TCP 헤더 20 B

어느 프로그램으로 갈지 적는 칸

보낸 포트 1057
받는 포트 80
seq 2039531092
ack 2775650109
신호 ACK+PSH
window 256

payload 97 B

실제 내용

GET /ncsi.txt HTTP/1.1
Connection: Close
User-Agent: Microsoft NCSI
Host: www.msftncsi.com

패킷은 겉 정보인 헤더와 속 내용인 payload로 구성되고, 본 연구는 payload 내용을 읽지 않음

같은 대화의 패킷을 한 flow로 묶기

flow = 두 컴퓨터의 두 프로그램이 같은 약속으로 주고받은 패킷 묶음, 즉 대화 한 건

프로토콜	IP A	포트 A	IP B	포트 B

다섯 값이 모두 같으면

같은 flow · 보낸 쪽과 받는 쪽이 바뀌어도 같은 flow

패킷	프로토콜	보낸 쪽	받는 쪽	묶인 결과
요청	**TCP**	**192.168.10.9 : 1057**	**69.31.33.224 : 80**	**flow A**
응답	**TCP**	**69.31.33.224 : 80**	**192.168.10.9 : 1057**	**flow A 방향만 반대**
인증 접속 1	TCP	192.168.10.9 : 1055	192.168.10.3 : 88	flow B
인증 접속 2	TCP	192.168.10.9 : 1056	192.168.10.3 : 88	flow C 임시 포트가 다름
도메인 조회	UDP	192.168.10.9 : 59285	192.168.10.3 : 53	flow D 프로토콜과 상대가 다름

묶는 이유 패킷 하나로는 행동을 알 수 없고, 대화 전체를 봐야 요청·응답·종료를 판단 가능

다섯 값이 같아도 대화가 끝나면 다음 패킷은 새 flow · 끝나는 조건은 다음 장

flow의 끝: FIN과 RST

→ PC가 보냄 ← 서버가 보냄 · 소속 칸이 각 패킷의 처리 · 두 대화 모두 월요일 실제 패킷

케이스 1 FIN으로 끝남 · 192.168.10.9:1057 ↔ 69.31.33.224:80

#	시각 (µs)	방향	TCP flag	길이	payload	소속
1	0	→	SYN	52	0	flow A
2	19,958	←	SYN+ACK	52	0	flow A
3	20,008	→	ACK	40	0	flow A
4	20,083	→	ACK+PSH	137	97	flow A
5	40,027	←	ACK	40	0	flow A
6	40,640	←	ACK+PSH	219	179	flow A
7	40,643	←	ACK+FIN	40	0	flow A
8	40,685	→	ACK	40	0	flow A
9	40,689	→	ACK+FIN	40	0	flow A
10	**60,574**	**←**	**ACK**	**40**	**0**	**flow A 마감**

케이스 2 RST로 끝남 · 192.168.10.51:38488 ↔ 209.15.36.31:80

#	시각 (µs)	방향	TCP flag	길이	payload	소속
1	0	→	SYN	60	0	flow B
2	42,246	←	SYN+ACK	60	0	flow B
3	42,350	→	ACK	52	0	flow B
4	5,079,690	→	ACK+FIN	52	0	flow B
5	5,084,903	←	ACK+FIN	264	212	flow B
6	**5,084,965**	**→**	**RST**	**40**	**0**	**flow B 마감**
7	**5,121,931**	**←**	**ACK**	**52**	**0**	**버림**
8	**5,121,962**	**→**	**RST**	**40**	**0**	**버림**

PC가 4번 FIN으로 먼저 닫아 도착하는 5·7번마다 RST로 응답

첫 RST까지가 한 flow · 뒤따르는 패킷은 앞 대화의 꼬리라 버림

TCP 신호

SYN 연결 요청 · ACK 받았다는 확인
PSH 내용 바로 전달 · FIN 보낼 것이 끝남
RST 연결을 버림, 확인 없이 즉시 종료

버리는 꼬리 패킷

닫힌 연결에서 2초 안에 도착
SYN·payload 없는 패킷 2개 이하

flow가 끝나는 때

RST · 양쪽 FIN 뒤 ACK · 같은 끝점의 새 SYN
120초 무통신 · 시작 후 120초
UDP·ICMP는 시간 조건만 사용

FIN 종료는 마지막 ACK까지, RST 종료는 첫 RST까지가 한 flow이고 뒤따르는 패킷은 버림

헤더 토큰에서 netFound feature로

flow의 패킷마다 헤더 값을 숫자 토큰으로 옮겨 netFound에 입력 · 아래는 케이스 1 flow A의 패킷

패킷	IP 헤더 길이	TOS	전체 길이	IP flag	TTL	TCP flag	window	seq 앞	seq 뒤	ack 앞	ack 뒤	urgent
1번 SYN	20	0	52	2	128	2	8192	0	0	0	0	0
2번 SYN+ACK	20	0	52	2	56	18	29200	0	0	0	1	0
4번 요청	20	0	137	2	128	24	256	0	1	0	1	0
…	…	…	…	…	…	…	…	…	…	…	…	…

flow 한 건의 헤더 토큰 · 패킷 1개 = 숫자 12개 · IP·포트·시각·payload 내용 제외

헤더 토큰

flow의 패킷 순서대로
간격·방향 함께 입력

netFound

사전학습 transformer
가중치 고정

PCA

512차원 → 64차원
학습 데이터로 맞춤

feature

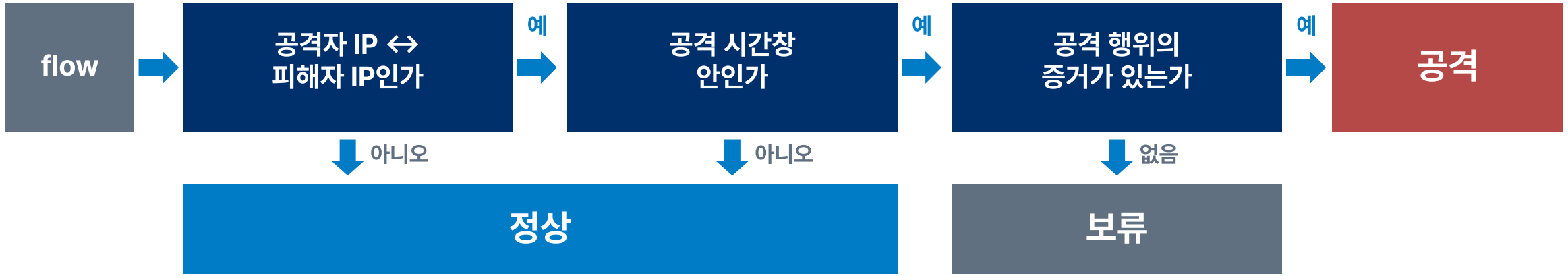
flow 한 건 = 64개
분류기의 한 행

netFound 네트워크 트래픽용 사전학습 transformer

라벨 없는 트래픽에서 가려진 헤더 토큰을 맞히며 학습, 본 실험은 인코더로만 사용

flow에 라벨 붙이기

각 flow를 왼쪽부터 판정 · CNS 2022의 교정 규칙 53개를 그대로 적용



라벨	조건	처리
공격	IP·시간창·증거를 모두 만족	학습·평가에 공격으로 사용
보류	IP·시간창은 맞지만 증거 없음	학습·평가에서 제외
정상	IP나 시간창이 규칙 밖	학습·평가에 정상으로 사용

증거 예 FTP·SSH brute force는 공격자 payload ≥ 1 B

증거 예 Portscan은 SYN 1개와 무응답 또는 RST

CNS 2022 Liu 외

Error Prevalence in NIDS Datasets

CIC-IDS 2017·2018의 flow 생성과
라벨 오류를 패킷 단위로 조사
교정한 라벨 규칙과 도구를 공개

IP·시간창·공격 행위 증거 세 조건으로 공격, 보류, 정상을 나누고 보류는 학습·평가에서 제외

최종 데이터 구성

잔여 flow 삭제 후 분할 전 기준 · CSV는 공식 라벨 행 수, PCAP은 라벨 있는 flow 수 · 행 순서는 PCAP flow 수 내림차순

class	세부 공격 (PCAP flow 수)	CSV 행	PCAP flow
benign	월요일 전일 + 공격일 배경	2,273,097	1,609,980
portscan	금 Portscan 161,202 · 목 NMAP 76,947	158,930	238,149
dos	Hulk 158,546 · GoldenEye 7,567 · Slowloris 3,859 · Slowhttptest 1,741	252,661	171,713
ddos	LOIC-HTTP	128,027	95,683
bf	FTP 4,021 · SSH 2,961	13,835	6,982
bot	Ares	1,966	736
web	Brute Force 73 · XSS 18 · SQL Injection 13	2,180	104
inf	Communication 32 · CoolDisk 4	36	36
heartbleed	Heartbleed	11	11
합계		**2,830,743**	**2,123,394**

CSV와 수가 다른 이유 flow 경계와 라벨 규칙 교정 · 목요일 NMAP은 CSV에서 BENIGN, 교정 규칙에서 portscan

inf는 학습 25개·test 6개, heartbleed는 학습 7개·test 2개로 표본이 매우 적음

문제1 재점검: 동일 feature·상이 라벨

잔여 flow 삭제 후 · feature가 같은데 class가 다른 flow · 패킷이 매우 적은 flow에 집중

feature	충돌 flow	비율
CSV 통계 78개	7,020	0.25%
netFound	**43,380**	**2.04%**

남은 충돌 Portscan 42,609 · 정상 770 · DoS 1

패킷 수	flow 수	입력 같은 %	충돌 flow
1	7,853	97.2	821
2	**505,950**	**97.2**	**42,443**
3~5	691,512	92.1	110
6~20	608,631	11.1	6
21 이상	309,401	0.8	0

원인

패킷이 매우 적은 flow는 숫자가 적고 대부분 고정값
예 스캔 SYN과 거절 RST, 패킷 2개

netFound가 입력 전에 버리는 정보
IP · 포트 · 시각 · payload 내용

추가 정보로 극복 가능

netFound 벡터 뒤에 열을 붙여 분류기에 입력, IP·포트 값은 넣지 않음

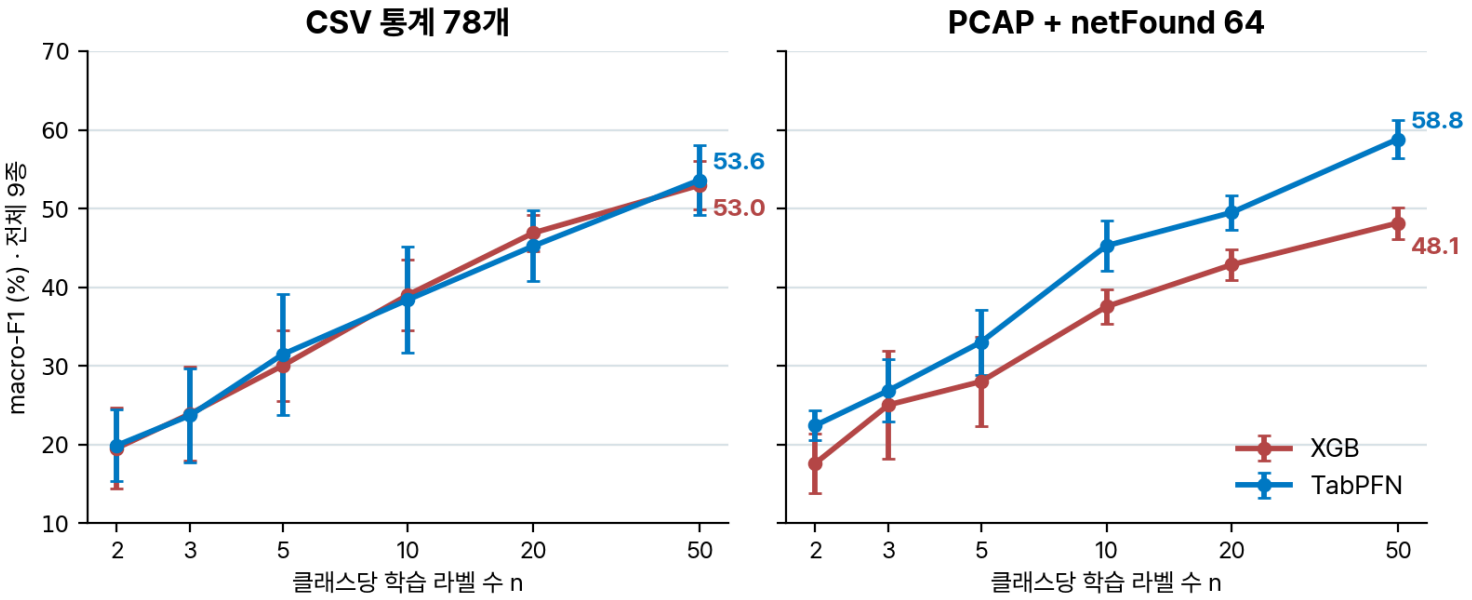
입력	충돌 flow	비율	감소
netFound	43,380	2.04%	–
+ 원본 seq 표시	42,571	2.00%	1.9%
+ 최근 60초 포트 수	964	0.05%	97.8%
+ 둘 다	**302**	**0.01%**	**99.3%**

최근 60초 포트 수 하나로 충돌 flow 97.8% 감소

남은 충돌 2.04%는 패킷이 매우 적은 flow와 netFound가 버리는 정보에서 발생, 추가 정보로 0.05%까지 감소

문제2 재점검: few-shot 성능

전체 9종 · 시간순 분할 · seed 5개 · 두 모델에 같은 학습 행 · netFound는 잔여 flow 삭제 데이터



macro-F1	XGB n=20	TabPFN n=20	XGB n=50	TabPFN n=50
CSV	46.9	45.2	53.0	53.6
netFound	42.9	49.5	48.1	58.8

CSV와 다른 점

CSV는 두 모델이 비슷
netFound는 TabPFN이 크게 우위
잔여 삭제 전후 7종 n=50 −0.5 pp

TabPFN이 우위인 이유 n=50, 삭제 전 측정

학습과 같은 입력의 정확도는 두 모델 모두 97.0%
나머지 행은 XGB 91.1% · TabPFN 97.5%
처음 보는 입력에서 격차 발생
PCA 벡터는 열 단위 분할에 불리 (추정)

netFound + TabPFN n=50 class별 F1

benign 97.9 · portscan 97.3 · dos 89.9 · ddos 99.1 · bf 94.6
bot 35.2 · web 10.0 · inf 0.2 · heartbleed 5.1
inf·heartbleed는 test 6개·2개, 재현율은 높고 정밀도 낮음

few-shot에서 netFound + TabPFN이 가장 높고, 격차는 학습과 겹치지 않는 입력에서 발생

추가 정보를 붙인 few-shot

전체 9종 macro-F1 · netFound 벡터 뒤에 열 추가 · seed 5개 · 같은 학습 행 · 잔여 flow 삭제 데이터

입력	XGB n=20	XGB n=50	TabPFN n=20	TabPFN n=50	val 차이 pp TabPFN n=50	정상 오탐 % TabPFN n=50	Portscan F1 XGB n=50
netFound	42.9	48.1	49.5	58.8	–	2.97	94.2
+ 원본 seq 표시	**43.5**	48.4	50.1	59.2	−0.4	3.18	94.9
+ 최근 60초 포트 수	43.1	**48.8**	**50.4**	**60.2**	**+2.3**	**2.48**	**98.5**

최근 60초 포트 수

TabPFN n=50 +1.4 pp, seed 5개 중 4개 상승
validation에서도 +2.3 pp

정상 오탐 2.97 → 2.48%
XGB Portscan F1 94.2 → 98.5

원본 seq 표시

TabPFN n=50 +0.4 pp, seed 5개 중 2개 상승
validation은 −0.4 pp

변화가 seed 편차 안에 있어 효과 불분명

최근 60초 포트 수는 validation과 test에서 소폭 개선, 원본 seq 표시는 효과가 불분명

정리와 다음 단계

문제1 동일 feature·상이 라벨

잔여 flow 삭제로 4.86 → 2.04%

남은 충돌은 패킷이 적은 스캔
60초 포트 수로 0.05%까지 감소

문제2 few-shot 성능

few-shot은 여전히 잘 동작
9종 n=50 macro 58.8%, CSV 53.6%

대량 공격은 소수 라벨로 포화
bot·web·inf는 여전히 낮음

PCAP 직접 가공의 장점

flow 경계와 라벨을 직접 교정

CSV에 없는 패킷 순서·맥락 사용

입력 구성을 실험 가능

앞으로

데이터	PCAP으로 가공할 수 있는 데이터 추가 탐색
모델	netFound feature로 SOTA와 제안 모델 실험
feature	최근 60초 포트 수를 입력에 포함해 모델 비교

PCAP을 직접 가공하면 데이터 문제를 고칠 수 있고, 다음은 데이터 확장과 SOTA·제안 모델 실험